

Financial Machine Learning: Foundations, Data Structures & Advanced Normality

Based on the Work of Marcos M. López de Prado

Graduate Seminar in Quantitative Finance (AI Finance Seminar)

Sogang Univ

September 2026

Table of Contents

- 1 Undergraduate Finance Foundations
- 2 Financial Data Structures
- 3 Graduate Advanced Normality & Stationarity
- 4 Conclusion & Summary

Section 1: Undergraduate Finance Foundations

Motivation for Non-Finance Majors

Machine learning algorithms are garbage-in, garbage-out. To prepare financial data properly, we must first understand the institutional and mechanical realities of the markets.

We will cover:

- **Financial Instruments:** Futures and ETFs
- **Market Microstructure:** Limit Order Books & Liquidity
- **Corporate Actions:** Splits, Dividends & Biases
- **Statistical Realities:** Why Time-based Sampling Fails

Financial Instruments: Futures & ETFs

- **Futures Contracts:**

- Agreements to buy/sell an asset at a predetermined future date.
- **Expiration & Roll-over:** Unlike stocks, futures expire. To build a continuous price series for ML, contracts must be joined together (rolled over), introducing price gaps and roll-over costs.

- **Exchange-Traded Funds (ETFs):**

- Baskets of securities traded on an exchange like single stocks.
- **Creation/Redemption:** Authorized Participants (APs) arbitrage the price gap between the ETF and its underlying basket, keeping prices close to Net Asset Value (NAV).

Market Microstructure: Limit Order Books & Liquidity

- **The Limit Order Book (LOB):**

- **Bids (Buy orders)** and **Asks (Sell orders)** represent resting liquidity.
- **Bid-Ask Spread:** The gap between the highest bid and lowest ask. This is the baseline transaction cost for crossing the spread.

- **Order Types:**

- **Limit Orders:** Passive, add liquidity, execute only at specified price or better, but suffer from non-execution risk.
 - **Market Orders:** Active, consume liquidity, execute immediately, but incur crossing costs.
- **ML Implication:** High-frequency trading and order splitting fragment orders into multiple small ticks, distorting simple tick-level metrics.

Corporate Actions & Financial Data Pitfalls

- **Corporate Actions:**

- **Stock Splits & Mergers:** Change share counts and nominal prices without changing firm value. Raw historical price series must be retroactively adjusted.
- **Dividends:** Cash payouts reduce the stock price on the ex-dividend date. Capital gains alone misrepresent total returns; we need *total return indexes*.

- **Critical Biases in ML Datasets:**

- **Look-ahead Bias (Backfilling):** Using future information (e.g., revised earnings, or stock split adjustments applied incorrectly in historical simulations) which was not known at the trade decision time.
- **Survivorship Bias:** Excluding bankrupt or delisted firms from historical backtests, artificially boosting model performance.

Statistical Quirks of Asset Prices: Why Time Bars Fail

The Default Standard: Time-based Sampling (e.g., 1-Min Bars)

Sampling prices at constant chronological intervals (hourly, daily) is the historical standard, but it is statistically toxic for Machine Learning.

- **Violations of IID Assumptions:** Financial returns sampled over time are highly non-normal, exhibiting fat tails (leptokurtosis) and serial correlation.
- **Heteroscedasticity:** Volatility and volume are highly seasonal (e.g., U-shape during trading hours). Chronological time ignores when information is actually entering the market.

Section 2: Financial Data Structures (AFML Chapter 2)

Chapter Goal

Transform unstructured market data (raw tick-by-tick transactions) into structured formats (Bars) that are optimal for Machine Learning.

Key concepts we will cover:

- Essential Financial Data Types (Fundamental, Market, Analytics, Alternative)
- Standard Bars (Time, Tick, Volume, Dollar)
- Information-Driven Bars (Imbalance & Runs)
- Dealing with Multi-Product Series (The ETF Trick)
- Event-Based Sampling (The CUSUM Filter)

The Four Essential Data Types

Data Type	Description	ML Challenges
Fundamental	Financial statements, macroeconomic	Low frequency, backfilling, revisions
Market	Price, volume, order book, LOB depth	High frequency, noise, size, non-IID
Analytics	Analyst ratings, news sentiment, signals	Subjective, proprietary, latency
Alternative	Satellite, credit cards, web traffic	Unstructured, sparse, expensive

Standard Data Bars

Standard bars aggregate raw tick data into discrete intervals based on a threshold:

- **Time Bars:** Sampled every t minutes. High heteroscedasticity.
- **Tick Bars:** Sampled every N transactions. Captures transaction frequency, partial correction for intra-day seasonality.
- **Volume Bars:** Sampled every V units/shares traded. Focuses on the size of the volume exchanged.
- **Dollar Bars (Highly Recommended):**
 - Sampled every D currency units (e.g., \$10M) exchanged.
 - **Robustness:** Strongest resilience to stock splits, massive price shifts, or share count adjustments.
 - Shows excellent statistical properties (distribution is much closer to normal/IID than other bars).

Empirical Demonstration: Time Bars vs. Dollar Bars

The Normality Simulation (Sogang Univ AI Finance)

Using a simulated 5-day tick dataset with a U-shaped intraday volatility pattern, we compare 10-Minute Time Bars vs. Dollar Bars (matched sample size).

Time-based Returns (10-Min)

- **Excess Kurtosis:** ≈ 1.85 (Fat tails)
- **Jarque-Bera p-value:** ≈ 0 (Strongly Rejected)
- **Q-Q Plot:** S-shape with extreme fat tails.
- **Cause:** Heteroscedasticity due to U-shaped intraday volume clustering.

Dollar-based Returns

- **Excess Kurtosis:** ≈ 0.20 (Near-Normal)
- **Jarque-Bera p-value:** ≈ 0.85 (Cannot Reject)
- **Q-Q Plot:** Excellent fit to the Gaussian line.
- **Mechanism:** Sampling frequency adapts dynamically to the information arrival rate.

Python Code: Simulating Normality Recovery

Key Steps in Python (scipy & seaborn)

```
# 1. Resample Time Bars (e.g., 10-Minute)
time_bars = df_ticks.resample('10Min').last().dropna()
time_rets = np.diff(np.log(time_bars['price']))

# 2. Resample Dollar Bars (using threshold)
# Sample when cumulative dollar value >= threshold
dollar_bars = df_ticks.iloc[bar_indices]
dollar_rets = np.diff(np.log(dollar_bars['price']))

# 3. Compute Excess Kurtosis and Test Normality
print(stats.kurtosis(time_rets))      # Output: ~1.85 (Highly Non-Normal)
print(stats.kurtosis(dollar_rets))   # Output: ~0.20 (Near-Normal!)
```

Information-Driven Bars: The Imbalance Concept

Designed to sample more frequently when **informed traders** enter the market, creating order flow imbalance. Let $b_t \in \{-1, +1\}$ be the trade sign via the tick rule:

$$b_t = \begin{cases} b_{t-1} & \text{if } \Delta p_t = 0 \\ \frac{|\Delta p_t|}{\Delta p_t} & \text{if } \Delta p_t \neq 0 \end{cases}$$

The cumulative imbalance up to tick T is $H_T = \sum_{t=1}^T b_t$ (or weighted by volume/dollars).

Imbalance Bars Condition

We sample a bar when the cumulative imbalance exceeds its expected value:

$$|H_T| \geq E_0[T \cdot |2P[b_t = 1] - 1|]$$

where $E_0[T]$ is the exponentially weighted moving average of the bar size (number of ticks), and the probabilities are updated dynamically.

Information-Driven Bars: The Runs Concept

While imbalance bars look at the net sign of the order flow, **Runs Bars** look at the sequence (run) of buy or sell trades. They capture the presence of **large institutional traders** who split their orders. Let $\theta_T = \max\left(\sum_{t|b_t=1} b_t, -\sum_{t|b_t=-1} b_t\right)$.

Runs Bars Condition

We sample a bar when the maximum run of buys or sells exceeds expectations:
 $\tau \geq E_0[T$

Dealing with Multi-Product Series: The ETF Trick

The Challenge

In ML, we often want to model a basket of assets or a spread. However, individual instruments change weight, pay dividends, have different trading hours, or roll over.

- **The ETF Trick:**

- A mathematical framework to represent a dynamic portfolio as a single, continuous, investable asset series.
- Accounts for transaction costs (crossing the bid-ask spread), rebalancing, cash drag, and dividends.
- Prevents look-ahead bias by strictly updating portfolio NAV based on lag-adjusted executed prices.
- Creates a clean, tradeable 1D time series of prices for the ML model to predict.

Feature Sampling: The CUSUM Filter

To train a machine learning model, we shouldn't sample features at every bar. We should only sample when there is a **structural event** or a regime change in the market.

Symmetric Cumulative Sum (CUSUM) Filter

An event-detector that identifies structural shifts in a series y_t (e.g., logged price):

$$S_t^+ = \max(0, S_{t-1}^+ + y_t - y_{t-1} - e)$$

$$S_t^- = \min(0, S_{t-1}^- + y_t - y_{t-1} + e)$$

We sample an event when $\max(S_t^+, -S_t^-) \geq h$ (threshold), and then reset the filters.

- **e**: The tolerance filter (drift parameter).
- **h**: The threshold parameter (can be scaled by rolling volatility).

Section 3: Graduate Advanced Normality & Stationarity

The Deep Mathematical Reality

Standard ML models assume IID and stationary data. But in finance, making data stationary usually destroys its historical memory, rendering it useless for long-term forecasting.

We will address:

- **Fractional Differentiation:** Achieving stationarity without losing memory.
- **Sequential Bootstrap:** Restoring sample independence under overlapping labels.
- **Random Matrix Theory:** Denoising and Detoning empirical covariance matrices.

Stationarity vs. Memory: The Ultimate Trade-off

Integer Differentiation ($d = 1$)

- Returns: $R_t = P_t - P_{t-1}$.
- **Good:** Stationary (ADF test passed).
- **Bad:** Complete memory loss.
High-frequency noise dominates.

No Differentiation ($d = 0$)

- Prices: P_t .
- **Good:** Maximum historical memory preserved.
- **Bad:** Highly non-stationary (spurious regressions, overfit models).

Solution: Fractional Differentiation ($0 < d < 1$).

Fractional Differentiation: Mathematical Formulation

Let B be the backshift operator ($B^k X_t = X_{t-k}$). The fractional differentiation of X_t with real degree d is:

$$(1-B)^d X_t = \sum_{k=0}^{\infty} (-1)^k \binom{d}{k} B^k X_t = \sum_{k=0}^{\infty} \omega_k X_{t-k}$$

where the weights ω_k are calculated recursively:

$$k = -\omega_{k-1} \frac{d-k+1}{k}, \quad \omega_0 = 1$$

Fixed-Width Window Fractional Differentiation (FFD)

To prevent infinite lag history requirements (which causes non-constant memory window size), the FFD drops terms where $|\omega_k| < \tau$ (a small tolerance threshold, e.g., 10^{-4}):

$${}_t = \sum_{k=0}^l \omega_k X_{t-k} \quad (\text{constant window width } l)$$

This yields a stationary series that retains more than 90% of the original price's correlation.

Sequential Bootstrap: Mitigating Label Overlap

- **The Overlap Problem:** Financial labels (e.g., Triple-Barrier Method) span multiple days/bars. If two labels share overlapping return windows, they are highly dependent (not IID).
- **Standard Bootstrap Fails:** Standard Bagging/Random Forest randomly samples rows with replacement, resulting in highly redundant out-of-sample sets and extreme overfitting.

Sequential Bootstrap Solution

Instead of uniform sampling, we compute the **Uniqueness** $u_i^{(t)}$ of a label at time t relative to the already selected sample set.

- ① Calculate the concurrency of active labels.
- ② Compute the uniqueness of each candidate sample.
- ③ Select the next sample with probability proportional to its uniqueness.

This restores sample independence, reducing out-of-sample generalization error.

Random Matrix Theory: Denoising Covariance Matrices

Empirical covariance matrices Σ calculated from short time-series are highly unstable and noisy, leading to catastrophic Markowitz portfolio optimization failures.

The Marcenko-Pastur Theorem

For a random matrix with dimensions $T \times N$, if $T, N \rightarrow \infty$ with $1 < \frac{N}{T} = q < \infty$, the eigenvalues λ of its correlation matrix follow:

$$f(\lambda) = \frac{q}{2\pi\sigma^2\lambda} \sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)} \quad \text{for } \lambda \in [\lambda_-, \lambda_+]$$

where $\lambda_{\pm} = \sigma^2(1 \pm \sqrt{1/q})^2$. Eigenvalues within $[\lambda_-, \lambda_+]$ represent pure noise.

• Denoising via Constant Residual Eigenvalues:

- Keep eigenvalues $\lambda > \lambda_+$ (Signal).
- Replace all eigenvalues $\lambda \leq \lambda_+$ (Noise) with their average value, keeping the trace (total variance) constant.

Random Matrix Theory: Detoning

The "Market Component"

The largest eigenvalue λ_{max} of a financial correlation matrix is typically massive, representing the market-wide dominant factor (all assets moving together).

- **The Detoning Process:**

- Financial clustering (unsupervised classification) can be blinded by this dominant market-wide component.
- Detoning removes the first eigenvector (associated with λ_{max}) from the correlation matrix:

$$\mathbf{C}_{detoned} = \mathbf{C} - \lambda_1 \mathbf{W}_1 \mathbf{W}_1^T$$

where $\mathbf{W}_1$ is the first eigenvector.

- **Result:** This allows algorithms to focus on sector-specific, micro-alpha relationships and subtler corporate clusters without market-wide bias.

Summary: The Financial ML Pipeline

To build a successful, scientific financial machine learning strategy, one must adhere to de Prado's production paradigm:

- ① **Institutional Grounding:** Build features with an understanding of microstructure (LOB, spreads, ETF trick).
- ② **Dollar Bars & CUSUM Sampling:** Move away from time-based bars to achieve statistical stationarity and normality.
- ③ **Fractional Differentiation:** Balance the trade-off between memory and stationarity (never use simple integer returns blindly!).
- ④ **Label Uniqueness:** Train ensembles using Sequential Bootstrap to prevent overfit.
- ⑤ **Denoised Covariances:** Never feed raw covariance matrices into portfolio optimizers. Apply Random Matrix Theory.

Q&A

Thank You!

Questions & Discussions